

A Peer-to-Peer Networking Protocol for Distributed Multiplayer Systems in Unity

Overview

The project will consist of building a peer-to-peer UDP based protocol, with an authoritative centralized server for finding lobbies/parties, and entirely peer-to-peer design for everything else, including joining/leaving the lobby, the internal logic, all edge cases: if a user lags or disconnects, and host migration. The goal is to make this protocol specific for video games, but we will try to make it as modular and generic as possible so at some point, we might reuse its base logic and its features for other peer-to-peer tasks (video conferencing, possibly messaging, distributed computing...). The game will be a top down 2d zombie shooter, where the player and up to 3 friends (4 player co-op) are placed on a map and shoot zombies with different weapons, with upgrades and waves of zombies which get progressively harder. Unity will be used as a game engine for any game logic, and the LiteNetLib will be used as a C# networking library, for its implementation of UDP, and reliable UDP.

Game

A player can start their own lobby, or join an existing one. The lobbies are indexed in the centralized server. The game is a top down 2D game, in the style of CS2D. When the game starts and the player has a few seconds to look around the map, find upgrades and weapons, the difficulty of the waves is determined by what wave it is, and the number of players in the game, for example if there are 4 players wave 1 is harder than if there are 3, 2 or 1. Between each wave the players have some time to look around and explore and get upgrades and weapons. Any player can leave or join the game at any point. Game runs at a fixed tick rate of 30 or 60 ticks per second (probably will be 30, to save on performance)

Networking

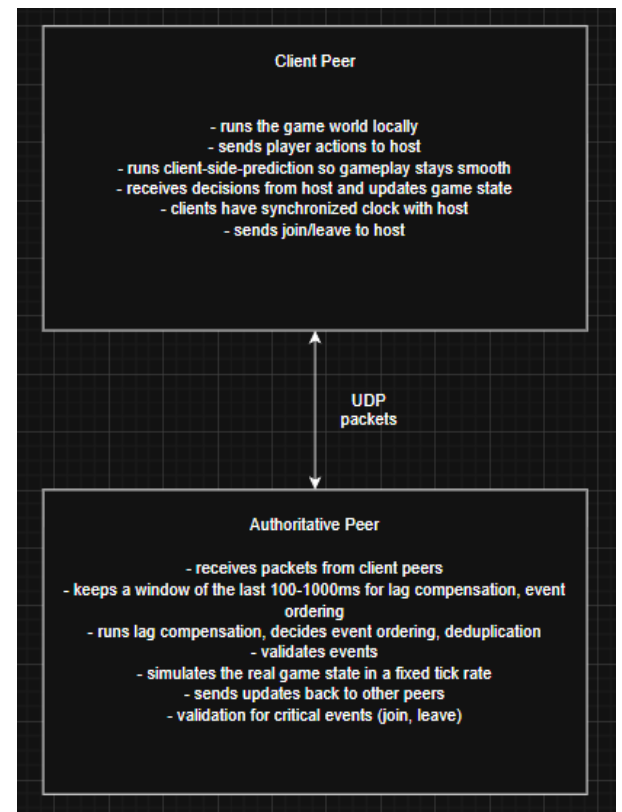
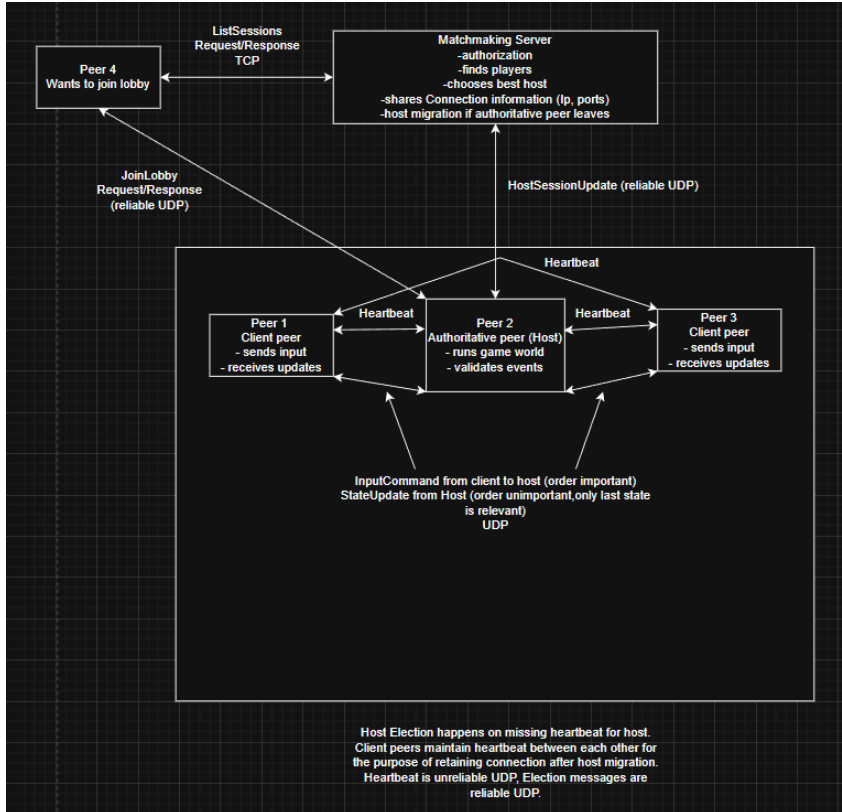
Most of the messages will be sent every tick and they are unreliable. But other messages have to be reliable, for that we will be using reliable UDP, as implemented in LiteNetLib, which adds sequence numbers, ACKs, retransmission timers, deduplication and reordering for UDP, but it is still faster than TCP because it is not stream based and does not contain a lot of the unnecessary bloat TCP has.

Message Types

- **Lobby and Connection:**
 - ListSessionsRequest (TCP) – client -> centralized server. Client asks for active sessions. Fields : {playerId (or empty)};

- ListSessionResponse (TCP) – centralized server -> client. Server returns list of available sessions. Fields: {sessions[] (sessionId, hostIp, hostPort, peerCount, latency, (possibly all peer:ip pair if host migration is going on, will see if it becomes issue when designing and testing))};
- HostSessionUpdate (reliable UDP MYB TCP) – host peer -> centralized server. Sent every 1000ms or when a player joins/leaves or unexpectedly disconnects, sends the central server accurate lobby information. Fields: {sessionId, hostIp, hostPort, hostName peerList[] (peerId,peerUsername, latency) };
- JoinLobbyRequest (reliable UDP) – player who wants to join -> host peer. Fields: {playerId, playerUsername, latency};
- JoinLobbyResponse (reliable UDP) – host peer -> player who want to join. Fields: {accepted, assignedPlayerPosition(or color, or some kind of order in the lobby)};
- LeaveLobby (reliable UDP) - client peer -> host peer. Message that player wants to leave. Fields: {playerId};
- **Game Events:**
 - InputCommand (ordered unreliable UDP) – client peer -> host peer. Sent every tick. Fields: {tick, cmdNumber, actions[] (move, shoot, pick up, open...)};
 - StateUpdate (ordered unreliable UDP) – host peer -> client peer. Sent every 500ms or after joining/leaving of a player, includes full world state... Fields: {tick, playerStates[], zombieStates[], worldState[] (what door is opened...)};
 - StateUpdateDelta (ordered unreliable UDP) – host peer -> client peers. Sent every tick, only the delta position of events (changes from the previous to the current state). Fields: {tick, playersStateDelta[], zombiesStateDelta[], worldStateDelta[]};
 - EventNotification (reliable UDP) – host peer -> client peer. Sent when important event happens (cutscene plays, game ends...). Fields: {tick, eventType, parameters[]};
- **Host Migration:**
 - Heartbeat (unreliable UDP) – peer -> peer (no matter if host or client). Send every 100-500ms between each peer. Fields: {tick, senderId};
 - HostFailure detect (reliable UDP) – client peer -> client peer. If no Heartbeat received from host peer in 5000ms (or longer period) initiate HostFailureDetect, peer that detects this sends it to every other peer every 1000ms. Fields {playerId, lastKnownHostTick};
 - HostElectionRequest (reliable UDP) – client peer -> client peer. Sent from peer with lowest overall latency (estimated with some algorithm) proposes to be the new host. Fields: {sessionId, candidateId, latency, candidateIp, candidatePort};
 - HostElectionResponse (reliable UDP) – client peer -> client peer. Other client peers respond that they have received the request and agree with the new host. Fields: {newHostId};

- HostClaim (reliable UDP) – new host peer -> client peer. Elected peer becomes host, game starts from its last known state, as known by the new host. Fields {playerId, snapshot (a specialized initial StateUpdate packet, is reliable)};



Other functionalities

Client-side prediction for smooth gameplay, for example if client shoots at a zombie or opens a door, it does not wait for a response to load the scene, because it would be too slow, it renders the events as the client sees them, and then when they receive an update from the host peer, if there is an error with the prediction, it gets seamlessly fixed (if the zombie was already dead by another player it gives the kill count to them).

Host keeps windows of event for the last 100-200ms (or longer), for accurate event reordering, deduplication and lag compensation (in some cases a player shoots at a zombie on their screen that isn't really there, but with lag compensation they will still get the kill to make it more seamless, depends on the case and the intensity of the lag).